Tarea 1

IIC3533 · Computación de Alto Rendimiento · 2026-2

Viernes 28 de Agosto de 2026

- Trabajo en grupos de tres personas.
- Los experimentos deben realizarse en al menos dos computadores distintos.
- **Plazo de entrega:** Viernes 25 de septiembre de 2026, 23:59.

Bootstrapping para regresión lineal

En regresión lineal se modela la relación entre k variables de entrada y una variable de salida y mediante:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k$$

Los coeficientes $\beta_0, \beta_1, \ldots, \beta_k$ son desconocidos y se estiman a partir de N observaciones. Para ello se construye la matriz de diseño $X \in \mathbb{R}^{N \times (k+1)}$, donde cada fila corresponde a una observación con un 1 antepuesto:

$$X = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_k^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_k^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & \cdots & x_k^{(N)} \end{bmatrix}$$

y el vector de salidas $\mathbf{y} = (y_1, \ldots, y_N)^\top$. La solución de mínimos cuadrados es:

$$\hat{\boldsymbol{\beta}} = (X^\top X)^{-1} X^\top \mathbf{y}$$

El método *bootstrap* permite estimar la incertidumbre de $\hat{\boldsymbol{\beta}}$ sin asumir una distribución teórica para el error. El algoritmo es el siguiente:

Paso 1. Calcular $\hat{\boldsymbol{\beta}}$ sobre el dataset completo.

Paso 2. Iterar sobre B *resamples*. Para cada $b = 1, \ldots, B$:

(I) Sortear N índices con reemplazo del conjunto $\{1, \ldots, N\}$. Un mismo índice puede aparecer más de una vez.

(II) Tomar las filas de X e $\mathbf{y}$ indicadas por esos índices, formando $X^{(b)} \in \mathbb{R}^{N \times (k+1)}$ e $\mathbf{y}^{(b)} \in \mathbb{R}^N$ (mismo tamaño que el original, pero con filas repetidas).

(III) Calcular $\hat{\boldsymbol{\beta}}^{(b)} = ({X^{(b)}}^\top X^{(b)})^{-1} {X^{(b)}}^\top \mathbf{y}^{(b)}$.

Paso 3. Construir el intervalo de confianza. Tras los B ajustes se tienen B vectores $\hat{\boldsymbol{\beta}}^{(1)}, \ldots, \hat{\boldsymbol{\beta}}^{(B)}$. Para cada coeficiente j, se ordenan los B valores $\hat{\beta}_j^{(1)}, \ldots, \hat{\beta}_j^{(B)}$ de menor a mayor y se descarta el 2,5 % inferior y el 2,5 % superior: el intervalo de confianza *bootstrap* al 95 % queda definido por los valores extremos.

Paralelismo de Tareas con `joblib`

En el algoritmo de bootstrapping, cada resample es **completamente independiente** de los demás, lo que permite ejecutarlos en paralelo. Este esquema se denomina *paralelismo de tareas*.

La librería `joblib`[1] facilita este patrón:

- Con `joblib.Parallel(n_jobs=p)`, se lanzan p procesos que ejecutan las tareas simultáneamente.
- Cada proceso es independiente a nivel del sistema operativo.
- El tiempo ideal con p procesos es $T(p) = T(1)/p$, aunque en la práctica existen costos adicionales (*overhead*) al crear procesos y distribuir el trabajo.

[1] `https://joblib.readthedocs.io`

Parámetros del experimento:

- $N = 100\,000$ observaciones.
- $k = 300$ variables de entrada, $B = 48$ *resamples*.
- Coeficientes verdaderos $\boldsymbol{\beta}^*$ generados aleatoriamente y conocidos.

(a) **[5 pts]** Genere los datos sintéticos del problema usando una semilla aleatoria fija:

(I) Muestree los $k+1$ coeficientes verdaderos $\boldsymbol{\beta}^*$ de forma independiente desde $\mathcal{N}(0,1)$.

(II) Genere una matriz de datos de N filas y k columnas con entradas i.i.d. $\mathcal{N}(0,1)$, y agréguele una columna de unos al inicio para formar $X \in \mathbb{R}^{N\times(k+1)}$.

(III) Calcule $\mathbf{y} = X\boldsymbol{\beta}^*$ y súmele un vector de N valores i.i.d. $\mathcal{N}(0,1)$ como ruido.

Los datos a utilizar para el problema serán:

$$X \in \mathbb{R}^{N\times(k+1)}, \qquad \mathbf{y} \in \mathbb{R}^{N}.$$

(b) **[10 pts]** Implemente las siguientes tres versiones del algoritmo de bootstrapping:

- `bs_auto.py`: usa `BaggingRegressor` de `sklearn` con `n_jobs=`p. El bootstrapping y el paralelismo son completamente internos.
- `bs_sklearn.py`: el paralelismo se implementa con `joblib.Parallel`, y el ajuste del modelo usa `LinearRegression` de `sklearn`.
- `bs_numpy.py`: el paralelismo se implementa con `joblib.Parallel`, y el ajuste resuelve $(X^\top X)\hat{\boldsymbol{\beta}} = X^\top \mathbf{y}$ usando solo NumPy.

Itere sobre cada versión y comente sus distintas mejoras con el propósito de reducir los tiempos. Es muy probable que su primera implementación no sea la más eficiente.

(c) **[10 pts]** Comente sobre la correctitud y reproducibilidad de sus resultados:

- Las tres versiones deben producir intervalos de confianza similares entre sí y consistentes con los coeficientes verdaderos $\boldsymbol{\beta}^*$.
- ¿En qué medida son equivalentes los resultados de las distintas versiones?
- ¿Qué condiciones se deben cumplir para que los resultados sean reproducibles entre ejecuciones?

(d) **[10 pts]** Explique cómo el *backend* `multiprocessing` de `joblib` crea los procesos en su sistema operativo:

- ¿Cómo se asigna memoria al crear un proceso?
- ¿Qué ocurre con los arreglos `X` e `y` al lanzar p procesos?

(e) **[10 pts]** Ejecute `bs_numpy.py` para distintos valores de p y observe la actividad del sistema usando el monitor de actividad de su sistema operativo (Monitor de Actividad en macOS, Monitor del Sistema en Linux). Adicionalmente, use `threadpool_info()` de `threadpoolctl` para inspeccionar cuántos threads utiliza NumPy internamente dentro de cada proceso.

- ¿Observa algún indicio de *oversubscription*?
- ¿Cómo se puede corregir?

Comente sus resultados.

(f) **[10 pts]** Declare el número de cores lógicos de su computador. Ejecute las tres versiones con $p \in \{1, 2, \ldots, p_{\text{máx}}\}$ procesos, donde $p_{\text{máx}}$ es el número de cores lógicos. Presente los tiempos de ejecución en función de p mediante un gráfico o tabla, comparando las tres versiones.

(g) **[15 pts]** A partir de los tiempos del ítem anterior, calcule para cada versión y cada p:

$$S(p) = \frac{T(1)}{T(p)}, \qquad E(p) = \frac{S(p)}{p}$$

- Grafique $T(p)$, $S(p)$ y $E(p)$ en función de p para las tres versiones, comparando con la curva ideal.
- Justifique su elección para $T(1)$.

- Comente los resultados.

(h) **[10 pts]** Calcule y grafique el *overhead* $T_o(p)$ en función de p. Identifique y comente las principales fuentes de *overhead* en este esquema.

(i) **[10 pts]** Usando la versión más eficiente del ítem (b), ejecute experimentos variando simultáneamente el número de procesos p y el número de threads internos t (`threadpool_limits`), con $p \cdot t \leq p_{\text{máx}}$. Grafique los tiempos de ejecución para cada combinación (p, t) y concluya cuál es la mejor.

(j) **[10 pts]** Compare los resultados obtenidos en los dos computadores utilizados. ¿Qué diferencias fueron las más notables? Justifique a qué se deben.

Configuración del entorno Python. Se recomienda el uso de **Miniconda** para crear un entorno e instalar los paquetes necesarios (Python 3.13, `numpy`, `joblib`, `threadpoolctl` y `matplotlib`):

```
conda create -n tarea1-hpc python=3.13 -y
conda activate tarea1-hpc
conda install numpy matplotlib joblib threadpoolctl -y
```

Instrucciones de entrega:

- Entregar un documento PDF con sus resultados a través de Canvas.
- El uso de IA está permitido, mientras esté debidamente declarado en el informe.
- Se evaluará la claridad y legibilidad de los gráficos y tablas. Es importante identificar títulos, *labels*, leyendas y tamaño de fuente adecuados.